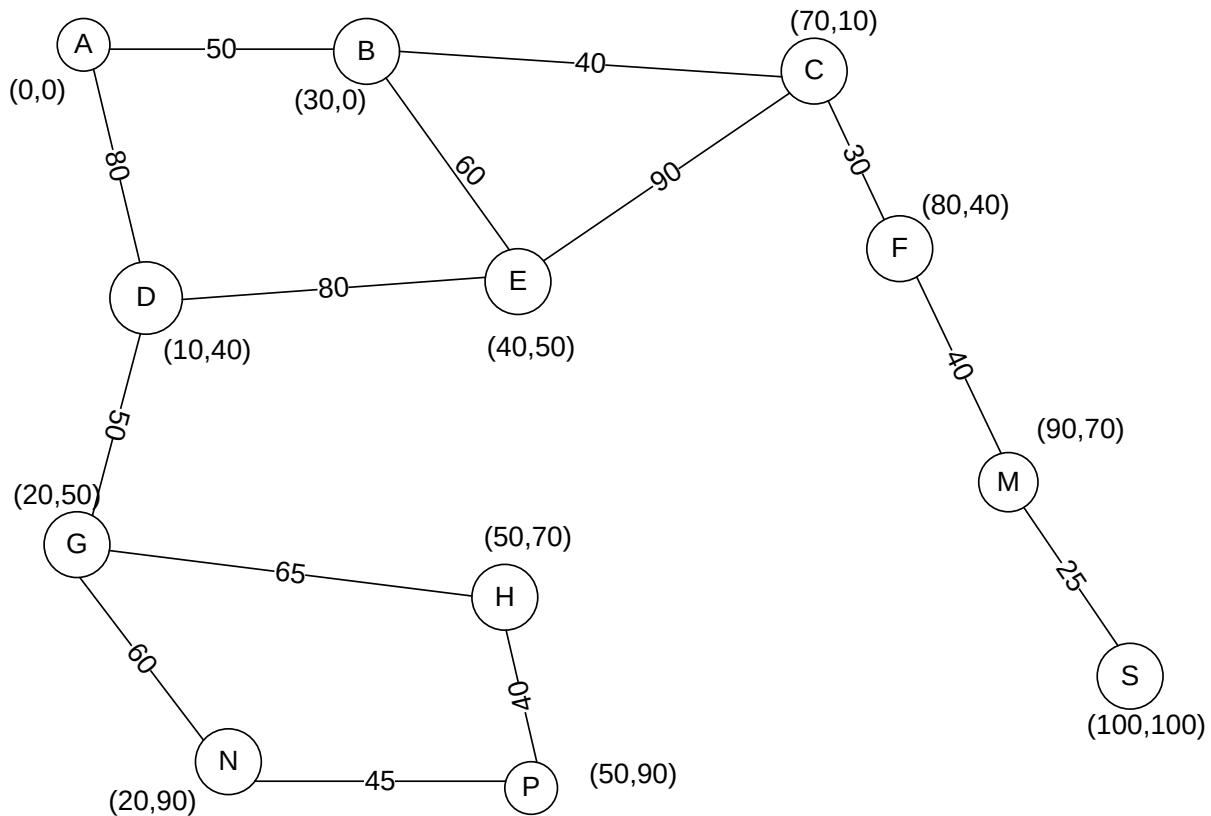


Artificial Intelligence

Search Algorithms

Assume a graph of cities is given as below.



Starting from city **A** we want to find the shortest path to city **S** (goal). The labels on the links show the distance between the cities. The coordinates of each city is available beside the nodes. In each step of the search we expand only one level so we can see the coordinates of the neighboring cities and the distance to each one of them.

To estimate the distance from a city to the goal use city block distance (the distance between $X(a,b)$ and $Y(c,d)$ is $|a-c| + |b-d|$. For instance the distance from $A1(10,20)$ to $A2(50,40)$ will be $(50-10) + (40-20) = 60$

Using the following algorithms determine the path to goal and discuss/compare the result of each method with the results of the other methods

1. Depth First Search: Use one branch. Continue until there is no node to visit, or the goal is found. If the goal is not found, go back one step and try another link. The links are selected in the order of storing them (pointer list)
2. Breadth First Search: Visit all child nodes at distance one from the initial state. Then visit all nodes at distance two, and so on, until either the goal is found or there is no node to continue.

Algorithm used by greedy and A*

Start with initial node. Put it in the list of visited nodes. Find the children of the nodes in the visited node list

Choose a child. Add it to the list of the visited nodes. Again find the children of the nodes in the visited nodes list and choose one

3. Greedy: In using the algorithm above, choose the child with shorter distance to the goal node
4. A*: In using the algorithm above, choose the child with shorter distance to (***initial node + goal node***)
5. Hill Climbing: Check the neighbors of the current node. Go the neighbor which is closer to the goal (should be also closer than the current node)

Deliverable:

For each algorithm find the path followed by the algorithm. Justify the path choice using the evaluation of the nodes (for instance, the distance of possible options as the next node to the goal)

A brief discussion on the results and your findings (which algorithm worked better, why)